

GitHub Proje Linki:

https://github.com/AFurkanOcel/Turkish_Multi_Level_Phase_Chunking

Turkish Multi-Level Phrase Chunking Using Classical Machine Learning

Bu raporda, Doğal Dil İşleme dersi kapsamında geliştirdiğim Türkçe çok seviyeli phrase chunking projesini ayrıntılı olarak sunuyorum. Projenin temel amacı, CoNLL formatında hazırlanmış Türkçe cümlelerde her token için OUTER_CHUNK, INNER_CHUNK ve CLAUSE etiketlerini otomatik olarak tahmin edebilen bir NLP sistemi geliştirmektir. Bu kapsamda yalnızca model eğitimi değil; veri seti hazırlama, etiketleme tutarlılığı kontrolü, özellik çıkarımı, model karşılaştırması, hata analizi, tahmin üretimi ve basit bir kullanıcı arayüzü geliştirme aşamaları da projeye dahil edilmiştir.

Doğal Dil İşleme (Natural Language Processing - NLP), insan dilinin bilgisayarlar tarafından işlenmesini ve anlamlandırılmasını amaçlayan bir çalışma alanıdır. Phrase chunking ise cümle içerisindeki anlamlı sözcük gruplarının belirlenmesini sağlayan temel NLP görevlerinden biridir. Bu görev sayesinde isim öbekleri, fiil öbekleri, zarf öbekleri ve cümle içindeki bazı yan yapılar token seviyesinde temsil edilebilir. Türkçenin eklemeli yapısı, fiilimsi kullanımları ve zengin morfolojik özellikleri nedeniyle bu görev, birçok dile kıyasla daha zorlayıcıdır.

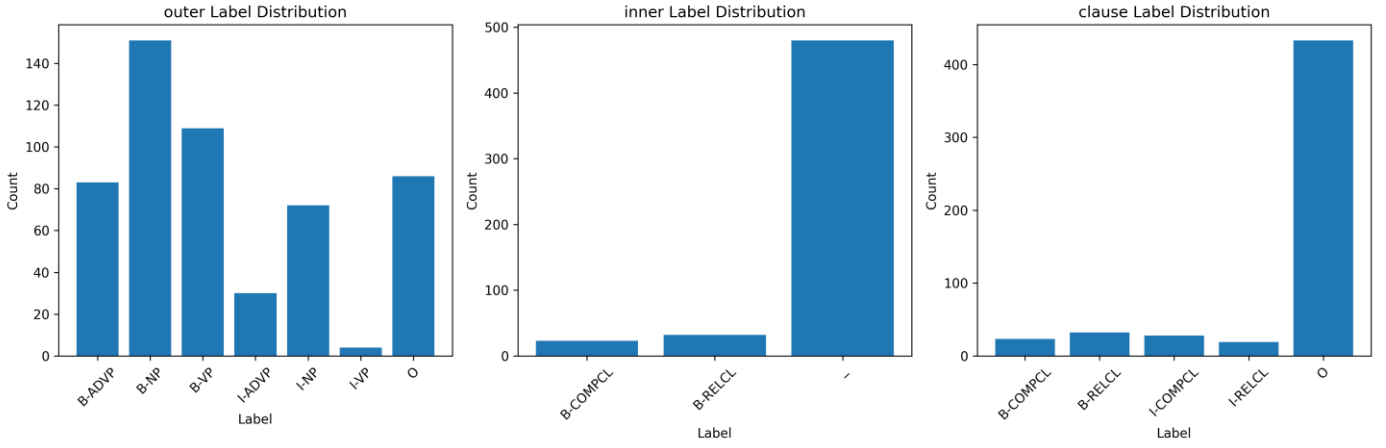
Bu çalışmada transformer, BERT veya büyük dil modeli tabanlı yöntemler kullanmadım. Bunun yerine scikit-learn kütüphanesi kullanılarak geliştirilen klasik makine öğrenmesi modellerini tercih ettim. Bu tercih hem proje gereksinimlerine uygun bir yaklaşım sunmakta hem de veri hazırlama, özellik çıkarımı, model davranışı ve hata analizi süreçlerinin daha açıklanabilir şekilde incelenmesine olanak sağlamaktadır. Bu nedenle proje, yalnızca yüksek metrik elde etmeye odaklanan bir uygulama değil; aynı zamanda Türkçe token-level sequence labeling probleminin veri, özellik ve model boyutlarıyla incelendiği uçtan uca bir çalışma olarak tasarlanmıştır.

CoNLL Formatı

ID FORM OUTER_CHUNK INNER_CHUNK CLAUSE

Dosya	Cümle	Token	Amaç
full_dataset.conll	100	679	Tüm veri seti
train.conll	80	535	Model eğitimi
test.conll	20	144	Model değerlendirme

Hedef	Etiketler
OUTER_CHUNK	B-NP, I-NP, B-VP, I-VP, B-ADVP, I-ADVP, O
INNER_CHUNK	B-RELCL, I-RELCL, B-COMPCL, I-COMPCL, _
CLAUSE	B-RELCL, I-RELCL, B-COMPCL, I-COMPCL, O



Şekil 1. Etiket dağılım grafiği

Şekil 1’de veri setindeki OUTER_CHUNK, INNER_CHUNK ve CLAUSE etiketlerinin dağılımı gösterilmektedir. Grafikte INNER_CHUNK için "_" etiketinin, CLAUSE için ise "O" etiketinin baskın olduğu görülmektedir. Bu sınıf dengesizliği nedeniyle model başarısını değerlendirirken yalnızca accuracy değil, F1-score ve confusion matrix sonuçları da dikkate alınmıştır.

1. Problem Tanımı ve Genel Çalışma Prensipleri

Phrase chunking, cümledeki anlamlı sözcük gruplarının token seviyesinde belirlenmesi işlemidir. Türkçe eklemeli yapısı, fiilimsi kullanımları ve esnek sözcük dizilişi nedeniyle bu görev açısından zorlayıcıdır.

Projede problem üç hedefe ayrılmıştır: OUTER_CHUNK ana phrase chunk etiketlerini, INNER_CHUNK daha dar yan yapı tetikleyicilerini, CLAUSE ise daha geniş yan cümlecik span bilgisini temsil eder. Genel akış: Data Loading -> Feature Extraction -> Training -> Evaluation -> Prediction -> Error Analysis.

2. Veri Seti ve Etiketleme Yapısı

Veri seti data/annotated klasörü altında full_dataset.conll, train.conll ve test.conll dosyalarıyla tutulmaktadır. Her satır bir tokeni temsil eder; boş satırlar cümle ayrımı, # ile başlayan satırlar yorum satırı olarak kullanılır.

Veri setini hazırlarken basit cümleler, zarf öbekleri, isim öbekleri, fiilimsi yapılar, relative clause, complement clause, bağlaçlı cümleler ve akademik/teknik örnekler kullanmaya dikkat ettim.

3. Veri Okuma ve Özellik Çıkarımı

src/data_loader.py dosyası, CoNLL formatını okuyup cümle yapısını korur. Hatalı satırlarda beklenen beş sütunu açıkça belirten hata mesajları üretir. Bu kontrol, model eğitiminden önce veri kalitesini yükseltir.

src/features.py içinde her token için word, lower, prefix, suffix, capitalization, digit, punctuation, token_length, previous_word, next_word, is_first_token ve is_last_token gibi özellikler çıkarılır. Türkçeye özgü suffix ipuçları da eklenerek fiilimsi ve clause yapılarının yakalanması hedeflenmiştir.

4. Kullanılan Modeller

Projede Majority Class Baseline, Multinomial Naive Bayes, Logistic Regression ve MLPClassifier modellerini karşılaştırdım. Baseline model, diğer modellerin gerçekten öğrenme yapıp yapmadığını görmek için referans olarak kullanıldı.

CRF modeli ana sisteme dahil edilmedi; önce temel klasik makine öğrenmesi modelleriyle çalışan sistemi tamamlamayı tercih ettim. CRF, gelecek çalışmalarda optional/bonus sequence labeling modeli olarak eklenebilir.

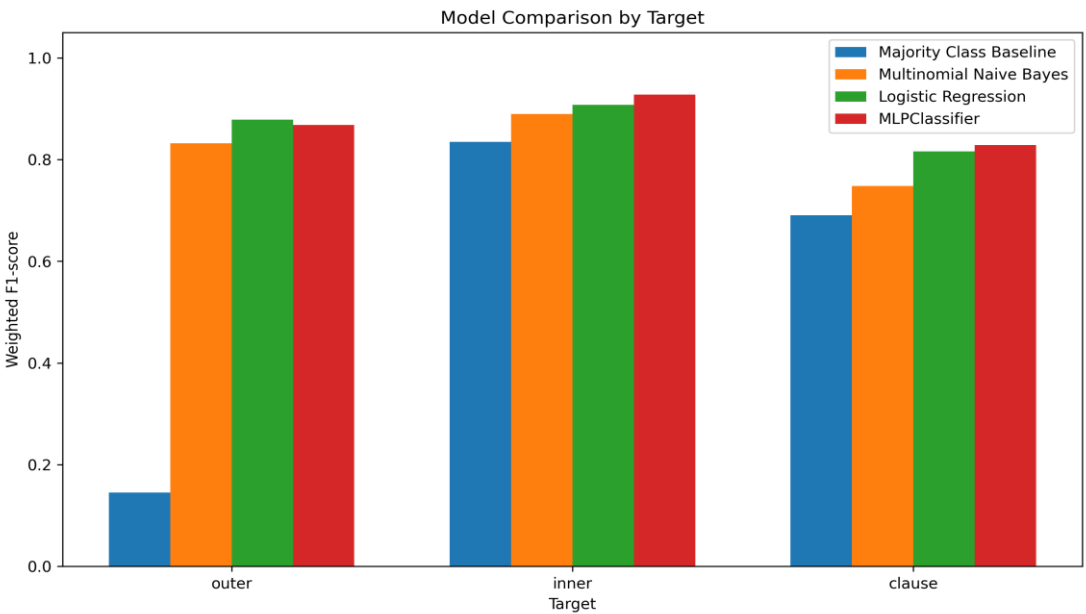
5. Eğitim, Değerlendirme ve Sonuçlar

Her hedef için ayrı model eğitildi ve sonuçlar accuracy, precision, recall ve F1-score metrikleriyle değerlendirildi. Classification report ve confusion matrix çıktıları, yalnızca genel başarıyı değil, sınıflar arası karışmaları da incelemeyi sağladı.

Sonuçlarda OUTER_CHUNK hedefinde Logistic Regression daha dengeli performans verirken, INNER_CHUNK ve CLAUSE hedeflerinde MLPClassifier en yüksek F1-score değerlerini elde etti.

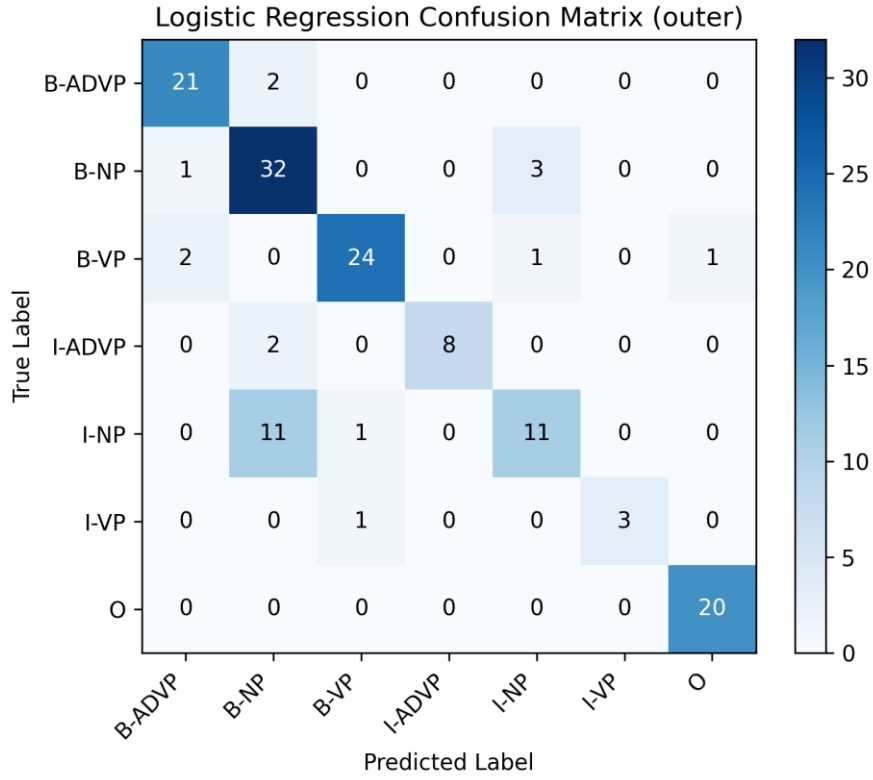
Hedef	En İyi Model	Accuracy	Precision	Recall	F1-score
OUTER_CHUNK	Logistic Regression	87.85%	88.46%	87.85%	87.83%
INNER_CHUNK	MLPClassifier	94.39%	91.92%	94.39%	92.80%
CLAUSE	MLPClassifier	85.98%	86.45%	85.98%	82.88%

Target	Model	Accuracy	Precision	Recall	F1-score
outer	Majority Baseline	30.84%	9.51%	30.84%	14.54%
outer	Naive Bayes	84.11%	85.14%	84.11%	83.19%
outer	Logistic Regression	87.85%	88.46%	87.85%	87.83%
outer	MLPClassifier	86.92%	87.62%	86.92%	86.83%
inner	Majority Baseline	88.79%	78.83%	88.79%	83.51%
inner	Naive Bayes	91.59%	88.21%	91.59%	88.98%
inner	Logistic Regression	92.52%	90.77%	92.52%	90.81%
inner	MLPClassifier	94.39%	91.92%	94.39%	92.80%
clause	Majority Baseline	78.50%	61.62%	78.50%	69.05%
clause	Naive Bayes	81.31%	75.18%	81.31%	74.80%
clause	Logistic Regression	85.05%	86.03%	85.05%	81.61%
clause	MLPClassifier	85.98%	86.45%	85.98%	82.88%



Şekil 2. Model comparison grafiği

Şekil 2 incelendiğinde Logistic Regression ve MLPClassifier modellerinin tüm hedeflerde Baseline ve Naive Bayes modellerine göre daha yüksek performans gösterdiği görülmektedir. Özellikle INNER_CHUNK görevinde MLPClassifier modeli en yüksek F1-score değerine ulaşmıştır.



Şekil 3. OUTER_CHUNK confusion matrix

Şekil 3'te OUTER_CHUNK görevi için elde edilen confusion matrix gösterilmektedir. Sonuçlar incelendiğinde modelin çoğu etiketi doğru tahmin ettiği, ancak bazı NP ve ADVP etiketlerinde karışıklıklar yaşandığı görülmektedir.

6. Tahmin Sistemi ve Streamlit Arayüzü

src/predict.py ile kullanıcı komut satırından Türkçe bir cümle girerek token bazlı OUTER_CHUNK, INNER_CHUNK ve CLAUSE tahminleri alabilir. Sonuç terminalde tablo olarak gösterilir ve outputs/predictions/predicted_sentence.conll dosyasına kaydedilir.

Streamlit arayüzü, sistemi daha sunulabilir hale getirir. Kullanıcı cümle girer, model seçer ve tahminleri tablo halinde görür. CSV export için index=False kullanıldığından gereksiz Unnamed: 0 sütunu oluşmaz.

```
python src/predict.py --sentence "Ayşe eve gelince hocanın önerdiği kitabı dikkatlice okudu."
```

Turkish Multi-Level Phrase Chunking

Model
Logistic Regression

Turkish sentence
Ayşe eve gelince hocanın önerdiği kitabı dikkatlice okudu.

Predict

	ID	FORM	OUTER_CHUNK	INNER_CHUNK	CLAUSE
0	1	Ayşe	B-NP	-	O
1	2	eve	B-NP	-	O
2	3	gelince	B-VP	B-COMPCL	I-COMPCL
3	4	hocanın	B-NP	-	O
4	5	önerdiği	I-NP	B-RELCL	O
5	6	kitabı	B-NP	-	O
6	7	dikkatlice	B-ADVP	-	O
7	8	okudu	B-VP	-	O
8	9	.	O	-	O

Download CSV

Predicted CoNLL output saved to: outputs/predictions/predicted_sentence.conll

Şekil 4’te geliştirilen Streamlit arayüzü gösterilmektedir. Bu arayüz sayesinde kullanıcı bir Türkçe cümle girerek seçilen model üzerinden OUTER_CHUNK, INNER_CHUNK ve CLAUSE tahminlerini token bazında tablo halinde görüntüleyebilmektedir.

7. Hata Analizi ve Veri Seti Tutarlılığı

Hata analizi dosyalarında sentence_id, token, gold_label, predicted_label ve context alanları bulunur. Bu dosyalar, modelin hangi tokenlarda ve hangi etiketlerde hata yaptığını görmeyi kolaylaştırır.

İlk denemelerde INNER_CHUNK ve CLAUSE hedeflerinin tek sınıfa düşme eğilimi gösterdiğini fark ettim. Bunun nedeni yalnızca model değil, veri setindeki dengesizlik ve bazı etiketleme tutarsızlıklarıydı.

Bu nedenle Dün, Bugün, sabah erken, dikkatlice, hızlıca ve düzgün bir şekilde gibi zaman/zarf ifadelerini bağımsız kullanımlarında ADVP olarak tutarlı hale getirdim. Bu ifadeleri gereksiz biçimde RELCL veya COMPCL kapsamına almadım.

RELCL kapsamını önerdiği, hazırladığı, gelen, başlayan gibi gerçek relative clause yapılarıyla sınırladım. COMPCL tarafında ise gelince, bitince, konuşurken, başlamadan, çıkınca ve ilerledikçe gibi complement/adverbial clause tetikleyicilerine odaklandım.

INNER_CHUNK ile CLAUSE etiketlerinin birebir aynı hale gelmesini engellemek için INNER_CHUNK alanını daha dar tetikleyici/predicate bölgesi, CLAUSE alanını ise daha geniş span olarak yorumladım. Bu veri tutarlılığı süreci model davranışını belirgin şekilde iyileştirdi.

8. Kısıtlar, Gelecek Çalışmalar ve Sonuç

Bu projenin temel kısıtı, veri setinin ders projesi ölçeğinde ve manuel hazırlanmış olmasıdır. Daha büyük ve doğal bir veri seti kullanılarak model sonuçları daha güvenilir şekilde değerlendirilebilir. Ayrıca veri setinin manuel etiketlenmiş olması nedeniyle insan kaynaklı etiketleme hatalarının tamamen ortadan kaldırılması mümkün olmamıştır.

Gelecekte CRF modeli, morfolojik çözümleyici, cross-validation ve daha ayrıntılı per-label hata analizi gibi geliştirmeler sisteme eklenebilir. Bunun yanında daha büyük veri setleri üzerinde çalışılarak farklı klasik makine öğrenmesi yöntemleri ve sequence labeling yaklaşımları da karşılaştırılabilir.

Sonuç olarak bu projede, Türkçe cümlelerde çok seviyeli phrase chunking işlemi gerçekleştiren uçtan uca çalışan bir NLP sistemi geliştirdim. Proje kapsamında veri setinin hazırlanması, veri okuma süreçleri, özellik çıkarımı, model eğitimi, performans değerlendirmesi, hata analizi ve kullanıcı arayüzü geliştirme aşamalarını tamamladım. Elde edilen sonuçlar, uygun özellik mühendisliği ile klasik makine öğrenmesi yöntemlerinin Türkçe phrase chunking görevlerinde başarılı şekilde kullanılabileceğini göstermektedir. Ayrıca proje sürecinde veri kalitesinin ve etiketleme tutarlılığının model performansı üzerindeki etkisini uygulamalı olarak gözlemleme fırsatı buldum.

Kaynakça

<https://aclanthology.org/anthology-files/pdf/W/W00/W00-0726.pdf>

<https://www.geeksforgeeks.org/nlp/what-is-conll-data-format/>

[https://en.wikipedia.org/wiki/Inside%E2%80%93outside%E2%80%93beginning_\(tagging\)](https://en.wikipedia.org/wiki/Inside%E2%80%93outside%E2%80%93beginning_(tagging))

<https://dergipark.org.tr/en/download/article-file/395201>

<https://github.com/agmmnn/turkish-nlp-resources>

<https://akgulomer.wordpress.com/2011/01/23/turkce-kelimelerin-morfolojik-analizi/>

<https://scikit-learn.org/stable/>

<https://docs.streamlit.io/>